

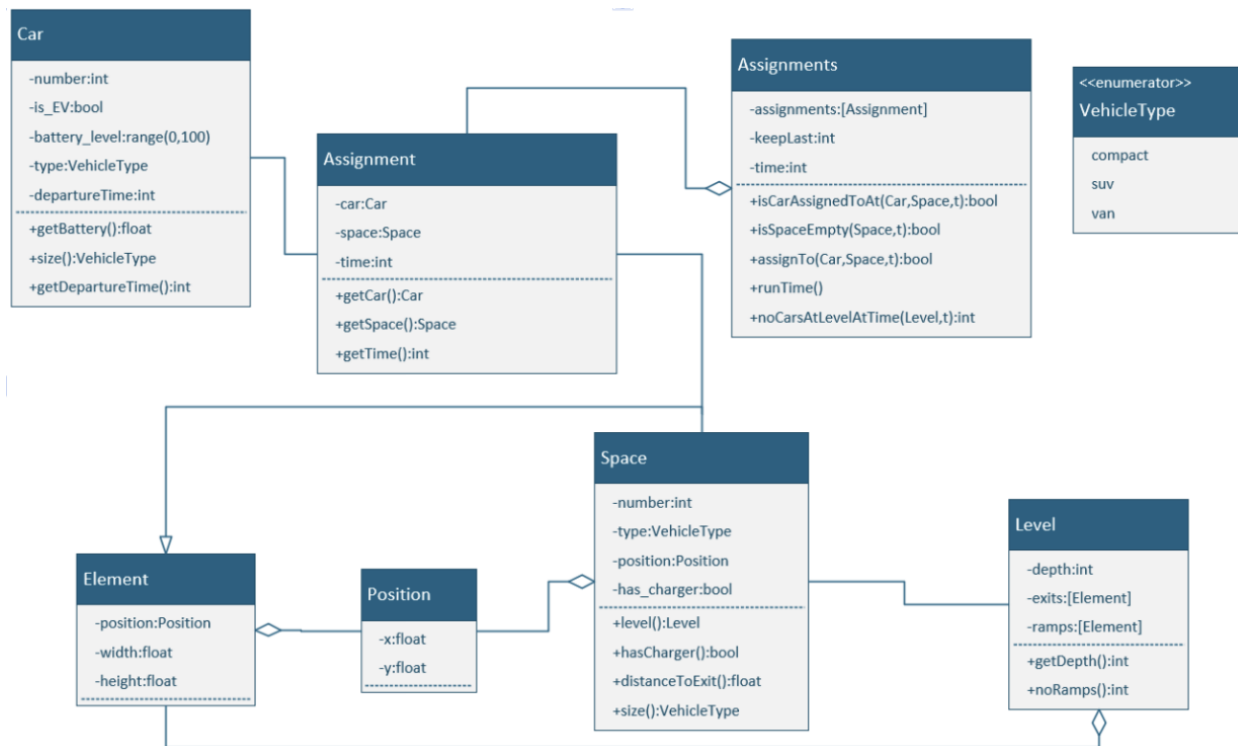


Smart Multi-Level Parking Optimization Problem

By Najm Al-Shamal

Benmalek Mohamed Aymen Horri Abderahmane Otsmane Mustapha Azizi Abdellah Esmail

UML Class Diagram



Problem Definition

Urban parking systems are becoming increasingly complex due to the growing number of vehicles and limited available space. Modern

parking facilities often consist of multiple levels with different capacities, accessibility constraints, and charging infrastructures for electric vehicles. Efficient allocation of incoming vehicles to available parking spaces is essential to reduce congestion, improve user experience, and optimize resource use. This problem focuses on management of a smart multi-level parking system where vehicles must be assigned to appropriate spots while considering spatial, temporal, and operational constraints.

Problem Statement

A multi-level parking facility receives a set of vehicles over time. Each vehicle must be assigned to exactly one available parking space upon arrival. Each space is located on a specific level and may have different characteristics (e.g., proximity to exits, electric charging availability, size compatibility). The objective is to design an optimal assignment strategy minimizing operational costs such as walking distance, congestion, and exit delay.

Objective Function

Our main objective is to maximize the average satisfaction rating of parking users.

Since vehicles arrive sequentially over time, parking decisions are made online: each arriving vehicle must immediately be assigned to one available parking space.

The assignment strategy is governed by a set of parking policies. Each policy is associated with a tunable weight representing its importance during the assignment process.

The considered policies include:

Policy	Description
Policy 1	Distance to exit
Policy 2	Congestion level
Policy 3	Parking level balance
Policy 4	EV charging preference
Policy 5	Size compatibility

For each arriving vehicle, all feasible parking spaces are evaluated using a weighted scoring function. The vehicle is then assigned to the parking space with the best score.

The main optimization problem therefore becomes:

Find the optimal policy weights that maximize the average user satisfaction.

Why Not Directly Optimize Individual Assignments

Since vehicles arrive one by one, selecting the locally best parking space for the current vehicle may negatively impact future users.

For example, first arriving vehicles may occupy the best parking spaces, forcing later vehicles into poor locations. Therefore, the objective is not only to optimize immediate assignments, but also to maintain fairness and balanced parking quality over time.

User Satisfaction Model

Because real user feedback data is unavailable, we define an artificial user rating model to approximate user satisfaction.

Different vehicle types prioritize different aspects of parking quality.

Compact Vehicle Users

Compact vehicle users generally prioritize:

- Short walking distance (High importance)
- Larger parking space (Positive impact)
- Congestion level (Medium importance)

SUV Users

SUV users generally prioritize:

- Short walking distance (High importance)
- Low congestion (High importance)
- Space compatibility (Important)

Van Users

Van users generally prioritize:

- Space compatibility (Critical)
- Low congestion (High importance)
- Short walking distance (Medium importance)

To simulate realistic human behavior, random noise is introduced into the rating model.

The total objective is defined as:

$$\text{Maximize TotalUserRating}$$

where:

$$\text{TotalUserRating} = \sum_{i \in \text{Vehicles}} \text{UserRating}_i$$

The optimization algorithms aim to maximize this total rating while satisfying all hard constraints.

Constraints

Hard Constraints

1. Each car must be assigned to exactly one parking place.
2. No space has 2+ cars at the same time.
3. Vehicle-size compatibility:
 - Van: only van places.
 - SUV: SUV or van places.
 - Compact: any place.

Soft Constraints: Policies

1. Electric vehicles should preferably be assigned to charging spots, especially at low battery.
2. Walking distance to exit should be minimized, considering level depth and internal distance.

3. Parking levels should remain balanced and not overcrowded.
4. Minimize congestion using the past k arrivals and departures at each level, including cars passing through.
5. Prefer matching car type to space size exactly — not smaller, not bigger.

First Formulation: Classical MILP

Sets

- Cars: $i \in C$
- Spaces: $j \in P$
- Levels: $l \in L = \{0, 1, 2\}$

Decision Variable

$X_{ij} \in \{0, 1\}$: 1 if car i is assigned to space j .

Parameters

Cars

- $B_i \in [0, 100]$: battery level (100 = non-electric).
- $S_i \in \{0, 1, 2\}$: size of car i .
- $\text{Arrival}_i, \text{Departure}_i$: time window.
- $I_{it} \in \{0, 1\}$: 1 if car i is present at time t .

Spaces

- $E_j \in \{0, 1\}$: charger available.
- $S_j \in \{0, 1, 2\}$: size of space j .
- $L_j \in L$: level of space j .

- $D_j \geq 0$: distance to exit.

Levels

- C_{lt} : number of cars at level l at time t .
- R_l : number of ramps at level l .

Hard Constraints

1. Each car must be assigned to exactly one parking place:

$$\sum_{j \in P} X_{ij} = 1, \quad \forall i \in C.$$

2. No space has 2+ cars at the same time.:

$$\sum_{i \in C} X_{ij} I_{it} \leq 1, \quad \forall j \in P \quad \forall t \in T.$$

3. Size compatibility:

$$X_{ij}(S_i - S_j) \leq 0,$$

$$\forall i \in C, j \in P.$$

Objective Function

The objective is to maximize the average user satisfaction across all assigned vehicles:

$$\max \quad \bar{R} = \frac{1}{N} \sum_{i=1}^N R_i$$

where $N = |C|$ is the number of vehicles and $R_i \in [1, 5]$ is the satisfaction rating of vehicle i .

User Rating Model

Each rating is a linear rescaling of a composite satisfaction score $S_i \in [0, 1]$:

$$R_i = 1 + 4 S_i$$

The score S_i aggregates five sub-scores weighted by the **policy weights** $w = (w_d, w_c, w_b, w_e, w_s)$ and **vehicle-type preference weights** α :

$$S_i = w_d \alpha_d^{(i)} D_i + w_c \alpha_c^{(i)} C_i + w_e \alpha_e^{(i)} E_i + w_s \alpha_s^{(i)} \Sigma_i$$

where each sub-score is clipped to $[0, 1]$.

Sub-Scores

Let vehicle i be assigned to space j :

1. Distance Satisfaction:

$$D_i = 1 - \frac{D_j}{D_{\max}}, \quad D_{\max} = \max_{j \in P} D_j$$

2. Congestion Satisfaction:

$$C_i = 1 - \frac{\text{occ}(L_j, t_i)}{\text{cap}(L_j)}$$

where $\text{occ}(l, t)$ counts vehicles on level l whose stay overlaps time t_i .

3. Level Balance:

$$B_i = 1 - \frac{|\text{occ}(L_j) - \bar{o}|}{\text{cap}(L_j)}, \quad \bar{o} = \frac{1}{|L|} \sum_{l \in L} \text{occ}(l)$$

4. EV Satisfaction:

$$E_i = \begin{cases} 1 & \text{if car } i \text{ is not EV,} \\ 1 & \text{if EV and } E_j = 1, \\ B_i/100 & \text{if EV and } E_j = 0. \end{cases}$$

5. Size Compatibility:

$$\Sigma_i = 1 - \frac{|S_i - S_j|}{2}$$

Vehicle-Type Preference Weights

Different vehicle types assign different importance to each criterion:

	α_d	α_c	α_b	α_e	α_s
Compact	0.35	0.15	0.10	0.05	0.35
SUV	0.30	0.30	0.10	0.05	0.25
Van	0.15	0.30	0.10	0.05	0.40

Formal MILP Objective

Combining everything, the full MILP objective is:

$$\max_X \quad \frac{1}{N} \sum_{i \in C} \sum_{j \in P} X_{ij} (1 + 4 S_i(j))$$

subject to the hard constraints defined above, where $S_i(j)$ denotes the composite satisfaction score when vehicle i is assigned to space j .

Exact Method: Branch and Bound for the Classical MILP

For the classical MILP formulation, the Branch and Bound algorithm seeks to **maximize the average user satisfaction rating** $\bar{R} = \frac{1}{N} \sum_{i=1}^N R_i$ over all N arriving vehicles.

Search Tree Structure

Each level of the search tree corresponds to one vehicle (sorted by arrival time), and each branch represents assigning that vehicle to a feasible parking space $j \in P$.

A node at depth d stores:

- The partial assignment of the first d vehicles.
- The accumulated satisfaction: $F_d = \sum_{q=1}^d R_q$.

Upper Bound

Since the maximum possible user rating is $R_{\max} = 5$, the optimistic upper bound at depth d is:

$$\text{UB}(d) = \frac{F_d + (N - d) \cdot 5}{N}$$

This assumes every remaining vehicle receives perfect satisfaction. The bound is admissible: $\text{UB}(d) \geq \bar{R}^*$ for any completion of the partial assignment.

Pruning Rule

Let $\bar{R}_{\text{best}}$ be the best average satisfaction found so far. The pruning condition is:

$$\boxed{\text{UB}(d) \leq \bar{R}_{\text{best}} \implies \text{prune subtree}}$$

If even the most optimistic completion cannot exceed the incumbent, the entire subtree is eliminated.

Branching and Node Ordering

At each node, feasible spaces are sorted in **descending order of local user rating** $R_i(j)$. This heuristic ordering ensures that promising assignments are explored first, leading to strong incumbents early and more effective pruning.

Algorithm

1. Sort vehicles by arrival time.
2. Initialize $\bar{R}_{\text{best}} = -\infty$.
3. Recursively explore the search tree:
 - If leaf: compute $\bar{R} = F_N/N$; update $\bar{R}_{\text{best}}$ if improved.

- If $\text{UB}(d) \leq \bar{R}_{\text{best}}$: prune.
- Otherwise: enumerate feasible spaces sorted by R_i descending, branch.

4. Return the assignment achieving $\bar{R}_{\text{best}}$.

Greedy Search Algorithm

Unlike the MILP formulation, where the objective is based on maximizing the overall user satisfaction ratings for all vehicles simultaneously, the greedy search algorithm operates in an on-line environment. Vehicles arrive sequentially, and the system must immediately assign a parking space without knowledge of future arrivals.

For this reason, directly maximizing the current user's rating may produce suboptimal long-term decisions. A parking assignment that highly satisfies the current user can negatively affect future users and reduce the global parking efficiency.

For example, assigning a compact vehicle to a large SUV parking space may increase the immediate satisfaction of the current driver because of the additional maneuvering space. However, if an SUV arrives afterward and no suitable large spaces remain available, the future user's satisfaction may decrease significantly. Similar issues arise for EV charging spaces, congestion-sensitive areas, or balanced level occupancy.

To address this limitation, the greedy algorithm uses a penalty-based evaluation system derived from the parking policies. Instead of directly maximizing user ratings, each candidate parking space is evaluated through policy penalties representing long-term operational costs and future parking quality preservation. The algorithm then selects the parking space that minimizes the global penalty score.

Penalty Terms

Let $i \in C$ be the arriving car and $j \in P$ the candidate space.

1. Battery / Charging:

$$P_{\text{Bat}} = (1 - E_j)((B_i - 100) \cdot \text{sc})^2$$

2. Walking Distance:

$$P_{\text{Dist}} = \mu(D_j + \nu L_j)$$

3. Level Balance:

$$P_{\text{Crowd}} = \text{Var}(C_{lt}) \cdot \text{sc}^2$$

4. Congestion ($0 < \Delta < 1$):

$$P_{\text{Cong}} = \sum_{\tau=T-k+1}^T \Delta^{T-\tau} \sum_{c,j} \max(X_{cj}^\tau, X_{cj}^{\tau-1})$$

$$\cdot \mathbb{1}_{l \leq L_j} \cdot |I_c^{\tau-1} - I_c^\tau| \cdot \frac{\text{sc}^3}{R_l}$$

5. Size Matching:

$$P_{\text{Size}} = |S_i - S_j|$$

Objective Function

$$\begin{aligned} \min Z = & \alpha P_{\text{Bat}} + \beta P_{\text{Dist}} + \gamma P_{\text{Crowd}} \\ & + \delta P_{\text{Cong}} + \lambda P_{\text{Size}} \end{aligned}$$

Genetic Algorithm for Weight Optimization

The greedy search algorithm depends on a set of policy weights:

$$\Theta = (\alpha, \beta, \gamma, \delta, \lambda)$$

where each parameter controls the importance of a parking policy in the objective function.

Selecting these weights manually is difficult because the policies interact with one another. Increasing the importance of one policy may improve some parking situations while degrading others. Furthermore, the optimal weights depend on traffic density, vehicle distributions, parking structure design, and user behavior.

To automatically discover high-quality policy weights, a Genetic Algorithm (GA) is used. The purpose of the GA is to evolve a set of weights that allows the greedy online algorithm to approximate the behavior of the offline MILP solution as closely as possible.

Chromosome Representation

Each chromosome represents one candidate set of policy weights:

$$\text{Chromosome} = (\alpha, \beta, \gamma, \delta, \lambda)$$

Each gene is a real-valued parameter:

$$\alpha, \beta, \gamma, \delta, \lambda \in \mathbb{R}^+$$

Fitness Function

For a given chromosome, the greedy algorithm is executed over the entire parking dataset using the corresponding weights.

The resulting parking assignments generate user satisfaction ratings:

$$R_i \in [1, 5]$$

The fitness value is defined as the total accumulated user satisfaction:

$$F(\Theta) = \sum_{i=1}^N R_i$$

where:

- N is the total number of arriving vehicles,
- R_i is the satisfaction score of vehicle i .

The objective of the GA is therefore:

$$\max_{\Theta} F(\Theta)$$

The MILP formulation has access to the complete dataset simultaneously and can therefore produce near-global optimal parking assignments. In contrast, the greedy algorithm only sees vehicles one by one. The role of the GA is to discover policy weights that compensate for this limitation and allow the online greedy strategy to converge toward the MILP behavior.

Population Initialization

An initial population of chromosomes is generated randomly:

$$\mathcal{P}^0 = \{\Theta_1, \Theta_2, \dots, \Theta_M\}$$

where M is the population size.

Each chromosome receives random positive values within predefined ranges:

$$\alpha, \beta, \gamma, \delta, \lambda \sim U(a, b)$$

Selection

At each generation, chromosomes with higher fitness values are more likely to be selected for reproduction.

Common selection strategies include:

- Roulette Wheel Selection,
- Tournament Selection,
- Rank Selection.

Tournament selection is preferred because it preserves diversity while remaining computationally efficient.

Crossover

Two parent chromosomes exchange genes to produce offspring.

Given:

$$\Theta_A = (\alpha_A, \beta_A, \gamma_A, \delta_A, \lambda_A)$$

$$\Theta_B = (\alpha_B, \beta_B, \gamma_B, \delta_B, \lambda_B)$$

an arithmetic crossover can produce:

$$\Theta_C = r\Theta_A + (1 - r)\Theta_B$$

where:

$$r \in [0, 1]$$

This allows offspring to inherit characteristics from both parents.

Mutation

To avoid premature convergence and maintain exploration, mutation randomly modifies genes:

$$\alpha' = \alpha + \epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

Mutation helps the algorithm escape local optima and explore unseen policy combinations.

Neighborhood Definition

The neighborhood of a chromosome consists of nearby weight configurations generated through small mutations:

$$\mathcal{N}(\Theta) = \{\Theta' : \|\Theta - \Theta'\| < \varepsilon\}$$

where:

- ε defines the neighborhood radius,
- neighboring chromosomes represent similar policy importance distributions.

The GA explores these neighborhoods iteratively in search of better parking strategies.

Elitism

To preserve the best solutions, elitism is applied by directly copying the highest-fitness chromosomes into the next generation.

If:

$$\Theta^* = \arg \max_{\Theta} F(\Theta)$$

then Θ^* is guaranteed to survive into the next population.

Stopping Criteria

The algorithm terminates when one of the following conditions is satisfied:

- Maximum number of generations reached,
- Fitness convergence achieved,
- No significant improvement observed for several generations.

Overall Objective

The final optimized weights:

$$(\alpha^*, \beta^*, \gamma^*, \delta^*, \lambda^*)$$

are then integrated into the greedy parking assignment algorithm to obtain an online strategy that approximates the global optimization quality of the MILP solution while remaining computationally efficient for real-time parking management.

Simulated Annealing for Weight Optimization

Similar to the Genetic Algorithm approach, Simulated Annealing (SA) is used to optimize the policy weights of the greedy parking assignment algorithm.

The objective is to determine the optimal weight vector:

$$\Theta = (\alpha, \beta, \gamma, \delta, \lambda)$$

such that the online greedy strategy produces parking decisions that approximate the quality of the offline MILP solution.

Unlike the MILP model, which has access to the complete dataset simultaneously, the greedy algorithm processes vehicles sequentially in real time. Therefore, carefully tuning the policy weights is essential to maintain high long-term user satisfaction.

Solution Representation

A solution state in the SA algorithm is represented by:

$$\Theta = (\alpha, \beta, \gamma, \delta, \lambda)$$

where each parameter controls the contribution of one parking policy in the greedy objective function.

Fitness Function

For a given solution Θ , the greedy parking algorithm is executed over the full dataset.

Each parked vehicle receives a satisfaction rating:

$$R_i \in [1, 5]$$

where:

- 1 represents very poor satisfaction,
- 5 represents excellent satisfaction.

The total fitness is defined as:

$$F(\Theta) = \sum_{i=1}^N R_i$$

where:

- N is the total number of vehicles,
- R_i is the rating of user i .

The optimization objective is:

$$\max_{\Theta} F(\Theta)$$

A higher fitness indicates that the selected weights generate parking decisions closer to the globally optimized MILP assignments.

Initial Solution

The algorithm begins with an initial randomly generated solution:

$$\Theta_0 = (\alpha_0, \beta_0, \gamma_0, \delta_0, \lambda_0)$$

with:

$$\alpha, \beta, \gamma, \delta, \lambda > 0$$

An initial temperature is also selected:

$$T_0 > 0$$

Neighborhood Structure

The neighborhood of a solution is generated through small perturbations of the weights.

Given:

$$\Theta = (\alpha, \beta, \gamma, \delta, \lambda)$$

a neighboring solution is:

$$\Theta' = (\alpha + \epsilon_1, \beta + \epsilon_2, \gamma + \epsilon_3, \delta + \epsilon_4, \lambda + \epsilon_5)$$

where:

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

and:

$$\|\Theta - \Theta'\| < \varepsilon$$

Small perturbations allow the algorithm to explore nearby policy configurations while preserving solution stability.

Acceptance Criterion

Let:

$$\Delta F = F(\Theta') - F(\Theta)$$

If:

$$\Delta F > 0$$

the new solution is accepted directly because it improves the fitness.

Otherwise, the solution may still be accepted with probability:

$$P = e^{\frac{\Delta F}{T}}$$

where:

- T is the current temperature,
- lower-quality solutions may temporarily be accepted to escape local optima.

Cooling Schedule

The temperature gradually decreases during the optimization process:

$$T_{k+1} = \eta T_k$$

where:

$$0 < \eta < 1$$

Typical values include:

$$\eta \in [0.8, 0.99]$$

At high temperatures, the algorithm explores broadly. As the temperature decreases, the search becomes increasingly focused around high-quality solutions.

Exploration and Exploitation

Simulated Annealing balances:

- exploration of new policy configurations,
- exploitation of high-performing weight combinations.

This balance is important because parking policy interactions are highly nonlinear. A small change in one weight may significantly affect future parking allocations and user satisfaction.

Stopping Criteria

The algorithm terminates when one of the following conditions is satisfied:

- temperature falls below a minimum threshold,
- maximum number of iterations reached,
- no significant fitness improvement observed.

Final Objective

The final optimized solution:

$$\Theta^* = (\alpha^*, \beta^*, \gamma^*, \delta^*, \lambda^*)$$

is integrated into the greedy parking assignment algorithm.

The resulting online strategy aims to preserve high user satisfaction while approximating the global optimization behavior of the offline MILP formulation with significantly lower computational complexity.

Second Formulation: SSP Model

Static Configuration

We assume a static configuration of l levels, denoted 0 to $l - 1$. Each level is an $n \times m$ array of parking spots, where spot (i, j, k) has x -coordinate i , y -coordinate j , and floor level k .

Every level shares the same $n \times m$ walking-distance matrix $W \in \mathbb{R}_{\geq 0}^{n \times m}$, where entry $W[i, j]$ is the Manhattan distance from spot (i, j) to the exit of that level, assuming all exits share the same planar coordinate:

$$W[i, j] = |i - i_{\text{exit}}| + |j - j_{\text{exit}}|.$$

Each parking spot carries a tuple of properties:

$(\text{size}, \text{ev_available}) \in \{\text{van}, \text{suv}, \text{compact}\} \times \{0, 1\}$.

State

A state s is a list of l matrices, each of size $n \times m$, where each cell carries the identity of the vehicle occupying it (or is empty):

$$s = (M^{(0)}, M^{(1)}, \dots, M^{(l-1)}), \quad M^{(k)} \in \mathcal{V}^{n \times m}.$$

The cell alphabet is:

$$\mathcal{V} = \{\emptyset\} \cup \{\text{van}, \text{suv}, \text{compact}\} \times \{0, 1\},$$

where $\emptyset$ denotes an empty spot, and a non-empty cell holds the pair $(\text{size}, \text{is_ev})$ of the parked vehicle. This gives $|\mathcal{V}| = 7$ possible values per cell. This enriched representation makes the quantities π_{ev} , N_{EV} , N_s , and π_s all directly computable from s .

Action

An action is a stochastic event governed by Poisson parameters λ (arrival rate) and μ (departure rate). There are two action types:

- **Arrival:** a vehicle requests a spot.
- **Departure:** a parked vehicle leaves.

Transition

Let p be the number of currently parked vehicles and $C = lnm$ the total cell count. At each step, exactly one event occurs:

Departure with probability $\frac{N\mu}{\lambda + N\mu}$. One occupied cell is chosen uniformly at random from the p occupied cells and flipped to $\emptyset$. There are p equally likely departure branches.

Arrival with probability $\frac{\lambda}{\lambda + N\mu}$. A vehicle is drawn with size distributed as $(\pi_{\text{van}}, \pi_{\text{suv}}, \pi_{\text{compact}})$ and type distributed as $\text{Bernoulli}(\pi_{\text{ev}})$, then the policy assigns it to one of the $a = C - p$ empty compatible cells. There are a possible arrival branches.

Such that N is the number of currently occupied spots.

Cost Function

The one-step cost of assigning a vehicle of size s_v to spot (i, j, k) — x -coordinate i , y -coordinate j , floor k — is:

$$\begin{aligned} \mathcal{C}(i, j, k) = & \pi_{\text{ev}} \cdot \frac{N_{\text{EV}}}{C_{\text{EV}}} + \sum_s \pi_s \cdot \frac{N_s}{C_s} \\ & + (\lambda + \mu) \frac{\text{cong}(k)}{l} \\ & + \frac{k}{l} + \frac{W[i, j]}{W_{\text{max}}} \\ & + \frac{s_v}{s_{\text{max}}} \cdot \text{crowd}(i, j, k) \end{aligned}$$

where the **accumulated congestion** up to floor k is defined as:

$$\text{cong}(k) = \sum_{q=0}^k \text{occ}(q),$$

with $\text{occ}(q) = \sum_{i,j} \mathbb{1}[M_{ij}^{(q)} \neq \emptyset]$ the total occupancy of floor q . This captures the fact that vehicles on higher floors must pass through all lower floors to exit, so congestion accumulates:

$$\begin{aligned} \text{cong}(0) &= \text{occ}(0) \\ \text{cong}(1) &= \text{occ}(0) + \text{occ}(1) \\ &\vdots \\ \text{cong}(l-1) &= \sum_{q=0}^{l-1} \text{occ}(q) \end{aligned}$$

The **local crowding** at spot (i, j) on floor k is defined as:

$$\text{crowd}(i, j, k) = \frac{\sum_{(i', j') \in \mathcal{N}(i, j)} \mathbb{1}[M_{i'j'}^{(k)} \neq \emptyset]}{|\mathcal{N}(i, j)|}$$

where $\mathcal{N}(i, j)$ is the set of grid-adjacent neighbors of (i, j) , with $|\mathcal{N}(i, j)|$ equal to 3, 5, or 8 for corner, edge, and interior spots respectively. The crowding penalty is weighted by $s_v/s_{\max}$, reflecting that larger vehicles are more affected by tight local packing.

where:

- π_{ev} : proportion of electric vehicles among currently parked vehicles (computed from s).
- C_{EV} : total number of EV-capable spots.
- N_{EV} : number of EVs *not* in EV-capable spots (computed from s).
- π_s : proportion of cars of size $s \in \{\text{van}, \text{suV}, \text{compact}\}$ among currently parked vehicles (computed from s).
- C_s : number of spots designated for size s .
- N_s : number of cars of size s *not* in size- s spots (computed from s).
- $\text{occ}(q)$: number of occupied spots on floor q .
- $W[i, j]$: Manhattan walking distance from spot (i, j) to the floor exit.
- $W_{\max}$: maximum value in the walking distance matrix.
- $s_v \in \{1, 2, 3\}$: size of the arriving vehicle (compact, SUV, van).
- $s_{\max} = 3$: maximum vehicle size.
- $\mathcal{N}(i, j)$: set of grid-adjacent neighbors of spot (i, j) on the same floor.

The terms penalise, in order:

- (1) EVs not parked in EV-capable spots.
- (2) Cars not parked in their designated size spots.

- (3) Arrival congestion on floors $0 \dots k$ (vehicles entering pass through lower floors).
- (4) Departure congestion on floors $0 \dots k$ (vehicles exiting pass through lower floors).
- (5) Floor level traversed upwards (preference for ground-level assignment).
- (6) Walking distance from the assigned spot to the floor exit.
- (7) Local crowding around the assigned spot, weighted by vehicle size.

Complexity Study

State Space

A state s is a tuple of l matrices, each of size $n \times m$, where each cell takes a value in the alphabet

$$\mathcal{V} = \{\emptyset\} \cup \{\text{van}, \text{suV}, \text{compact}\} \times \{0, 1\},$$

giving $|\mathcal{V}| = 7$ possible values per cell (one empty value, plus three sizes times two EV types). The total number of cells is $C = l \cdot n \cdot m$, so the state space cardinality is:

$$|\mathcal{S}| = 7^{l \cdot n \cdot m}$$

This grows **exponentially** in all three dimensions l , n , and m . To illustrate:

l	n	m	$ \mathcal{S} $
2	3	3	$7^{18} \approx 1.6 \times 10^{15}$
3	4	5	$7^{60} \approx 4.2 \times 10^{50}$
4	5	5	$7^{100} \approx 3.2 \times 10^{84}$

Even for the smallest configuration, the state space vastly exceeds what can be enumerated. This is a direct consequence of storing vehicle identity (size and EV status) in each cell rather than a simple occupancy bit. This phenomenon is known as the **curse of dimensionality**.

Action Space

At each decision epoch, exactly one event occurs. There are two event classes:

- **Departure:** any one of the p currently parked vehicles may leave, yielding p possible departure outcomes.
- **Arrival:** the arriving vehicle may be assigned to any of the $C - p$ currently empty cells, yielding $C - p$ possible assignment actions.

The total number of transitions from any state is therefore:

$$|\mathcal{A}(s)| = p + (C - p) = C = l \cdot n \cdot m$$

The action space grows **linearly** in the number of cells — it is not the computational bottleneck. The state space is.

Transition Structure

The transition function is stochastic. From any state s with p occupied cells, the next state is determined by:

- A **departure** with probability $\frac{N\mu}{\lambda + N\mu}$, uniformly selecting one of p occupied cells to empty.
- An **arrival** with probability $\frac{\lambda}{\lambda + N\mu}$, assigning the incoming vehicle (drawn from the (π_s, π_{ev}) distributions) to one of $C - p$ empty cells according to the policy.

The full transition matrix has size $|\mathcal{S}|^2 = 7^{2C}$. Storing and computing this matrix exactly is entirely infeasible for any non-trivial instance, ruling out pre-computation of the optimal value function $V^*(s)$ via value iteration.

Problem Complexity

The assignment subproblem — assigning N vehicles to N spaces subject to size compatibility, EV compatibility, and capacity constraints while minimizing total walking distance and congestion — is a variant of the **Generalized Assignment Problem (GAP)**, which

is known to be $\mathcal{NP}$ -hard [?]. The introduction of stochasticity (random arrivals and departures governed by a CTMC) and the multi-level structure (cascading ramp congestion) make the full problem strictly harder. We therefore classify the Smart Multi-Level Parking Optimization Problem as $\mathcal{NP}$ -hard.

Exact Method: Branch and Bound for the SSP Formulation

In contrast to the classical MILP formulation, which maximizes user satisfaction ratings, the SSP formulation seeks to **minimize the total assignment cost** $\sum_i \mathcal{C}_i$ accumulated over the sequence of vehicle arrivals.

Search Tree Structure

The Branch and Bound algorithm explores the space of all possible assignments as a tree. Each level of the tree corresponds to one arriving vehicle (sorted by arrival time), and each branch at that level represents assigning that vehicle to one of the feasible parking spots.

A **node** at depth d represents:

- A partial assignment of the first d vehicles: $\{v_1 \rightarrow (i_1, j_1, k_1), \dots, v_d \rightarrow (i_d, j_d, k_d)\}$.
- The accumulated cost so far: $G_d = \sum_{q=1}^d \mathcal{C}(i_q, j_q, k_q)$.

A **leaf node** at depth $N = |C|$ represents a complete assignment of all vehicles.

Branching Strategy

At each node for vehicle v_d , the algorithm enumerates every feasible spot (i, j, k) satisfying:

- Size compatibility:** $\text{size}(v_d) \leq \text{size}(i, j, k)$ under the ordering $\text{compact} < \text{SUV} < \text{van}$.
- Temporal availability:** the spot has no other assigned vehicle whose stay overlaps $[\text{arrival}_{v_d}, \text{departure}_{v_d})$.
- EV compatibility** (optional): if en-

forced, EV vehicles may only be assigned to EV-capable spots.

For each feasible spot, the one-step cost $\mathcal{C}(i, j, k)$ is computed from the SSP cost function using the current state, then a child node is created with accumulated cost $G_{d+1} = G_d + \mathcal{C}(i, j, k)$.

Node Ordering

Before branching, feasible spots are sorted in **ascending order of assignment cost**. This ensures that the most promising (lowest-cost) assignments are explored first, allowing the algorithm to find good incumbents early and prune more aggressively.

Bounding and Pruning

Since all cost terms are non-negative:

$$\mathcal{C}(i, j, k) \geq 0 \quad \forall (i, j, k),$$

the accumulated cost G_d at any partial node is a valid **lower bound** on the total cost of any complete assignment extending this node:

$$\text{LB}(d) = G_d \leq G_d + \sum_{q=d+1}^N \mathcal{C}(i_q, j_q, k_q) = G_N.$$

Let G^* denote the cost of the best complete solution found so far (the incumbent). At every node, the pruning condition is:

$$G_d \geq G^* \implies \text{prune subtree}$$

Any subtree rooted at a node whose accumulated cost already meets or exceeds the incumbent cost cannot possibly yield a better solution and is therefore pruned entirely. This is a strict pruning rule enabled by the non-negativity of the SSP cost function.

State Reconstruction

At each node, the algorithm reconstructs the occupancy state at the arrival time of the current vehicle:

$$\text{state}(t) = \{(i, j, k) : \exists v \text{ assigned to } (i, j, k) \text{ with arrival } \leq t \text{ and departure } > t\}$$

This state is used to compute all state-dependent cost components: floor occupancy

$\text{occ}(q)$, accumulated congestion $\text{cong}(k)$, local crowding $\text{crowd}(i, j, k)$, EV mismatch N_{EV} , and size mismatch N_s .

Algorithm Summary

1. Sort vehicles by arrival time.
2. Estimate Poisson parameters λ and μ from the traffic data.
3. Precompute the walking-distance matrix W and W_{max} .
4. Initialize incumbent $G^* = +\infty$.
5. Recursively explore the search tree:
 - If leaf: update G^* if $G_N < G^*$.
 - If $G_d \geq G^*$: prune.
 - Otherwise: enumerate feasible spots sorted by cost, branch recursively.
6. Return the assignment achieving G^* .

Complexity

In the worst case, the search tree has depth N and branching factor up to $C = l \cdot n \cdot m$ at each level, giving:

$$\mathcal{O}(C^N)$$

nodes. The pruning mechanism and cost-ordered branching drastically reduce the effective search space on practical instances, but the worst-case complexity remains exponential. This is expected: the underlying Generalized Assignment Problem is $\mathcal{NP}$ -hard, and no exact method can avoid exponential worst-case behavior unless $\mathcal{P} = \mathcal{NP}$.

Metaheuristics:

Local Search: Greedy Search

Given the intractability established above, we forgo exact methods in favor of a one-step lookahead greedy local search. The algorithm operates directly on the SSP state s and, at each arrival event, selects the assignment action that minimises the *expected* one-step cost over all stochastic outcomes reachable from the current state.

Why One-Step Lookahead?

Evaluating even two steps ahead requires marginalising over the joint distribution of the next event type (arrival vs. departure), the incoming vehicle’s type and size, and the specific departure branch — yielding a branching factor of $O(C^2)$ per state and an exponential tree in depth. More critically, looking ahead by d steps requires choosing a *fixed* assignment for intermediate states, which is itself the hard sub-problem we cannot solve exactly. Restricting to depth $d = 1$ keeps evaluation tractable while preserving the stochastic structure of the model: instead of collapsing to a deterministic look at the immediate successor, we take the expectation over all possible next states.

Expected One-Step Cost

At an arrival decision epoch, the system is in state s with p occupied cells. The algorithm enumerates every empty compatible cell $c = (i, j, k)$ as a candidate assignment. For each candidate, it constructs the hypothetical next state s_c obtained by placing the arriving vehicle at c , and evaluates the **expected cost** of s_c :

$$\mathbb{E}[\mathcal{C} \mid s_c] = \frac{N\mu}{\lambda + N\mu} \cdot \mathcal{C}_{\text{dep}}(s_c) + \frac{\lambda}{\lambda + N\mu} \cdot \mathcal{C}_{\text{arr}}(s_c)$$

where the two terms correspond to the two next-event branches:

- **Departure branch** (probability $\frac{N\mu}{\lambda + N\mu}$): one of the $p + 1$ occupied cells in s_c is freed uni-

formly at random. The cost $\mathcal{C}_{\text{dep}}(s_c)$ averages over all $p + 1$ possible departure outcomes.

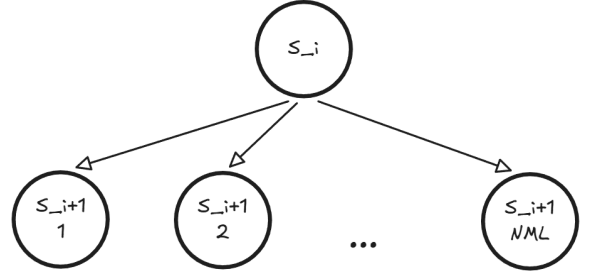
- **Arrival branch** (probability $\frac{\lambda}{\lambda + N\mu}$): a new vehicle arrives with size drawn from $(\pi_{\text{van}}, \pi_{\text{suV}}, \pi_{\text{compact}})$ and EV status from $\text{Bernoulli}(\pi_{\text{ev}})$. The cost $\mathcal{C}_{\text{arr}}(s_c)$ is the expectation of $\mathcal{C}(\cdot)$ over this vehicle-type distribution, evaluated at the best available empty cell in s_c (or zero if the lot is full).

The policy then selects the assignment c^* that minimises this expectation:

$$c^* = \arg \min_{c \in \text{compatible empty cells}} \mathbb{E}[\mathcal{C} \mid s_c].$$

Preserving the Stochastic Structure

The nature of this strategy ensures we search only 1 depth ahead, hence the greedy name. The reason is that going beyond one depth introduces deterministic assumptions about the occupation of the parking



Comparative Analysis

This section evaluates the performance of the proposed parking assignment approaches:

- Offline Classical MILP Optimization (Branch & Bound),
- State-Space SSP Formulation (Branch & Bound),
- Online Greedy Search,
- Online Random Assignment (Seed = 42),
- Greedy Search + Genetic Algorithm (GA-optimized weights),

- Greedy Search + Simulated Annealing (SA-optimized weights).

The objective is to compare solution quality, computational efficiency, scalability, and user satisfaction under identical parking datasets.

Experimental Setup

The experiments were conducted using the following configuration:

- Number of parking spaces: **48 spaces** (arranged as 3 levels with 16 spots per level)
- Number of parking levels: **3 levels** (with physical entrance/exit on Level 1)
- Vehicle categories: **Compact, SUV, Van, and EV (Electric Vehicles)**
- Simulation scenarios:
 - **Scenario 1 (Low traffic)**: 6 vehicles over a 150-second timeline.
 - **Scenario 2 (Medium traffic)**: 23 vehicles over a 600-second timeline.
 - **Scenario 3 (High traffic)**: 44 vehicles over a 1200-second timeline.
- Optimization timeouts: **MILP and SSP B&B are limited to 60 seconds.**

The same datasets were used across all algorithms to ensure a fair comparison.

Evaluation Metrics

The following metrics were used for evaluation:

- Average user satisfaction rating (scaled in $[1.0, 5.0]$),
- Total accumulated satisfaction rating,
- Parking utilization rate (% of distinct spaces occupied),
- Runtime execution (seconds),
- Assignment optimality gap relative to Classical MILP.

Comparative Results

The tables below present the performance of the six methods under different traffic intensities.

Scenario 1: Low Traffic (6 Vehicles, 48 Spaces)

Method	Avg Rating	Total Rating	Assigned	Util %	Runtime	MILP Gap
MILP (B&B)	4.8496	29.10	6/6	15.2%	2.9881s	0.00%
SSP (B&B)	4.1958	25.17	6/6	15.2%	60.0011s	13.48%
Greedy Search	4.7462	28.48	6/6	15.2%	0.0044s	2.13%
Random	3.8806	23.28	6/6	18.2%	0.0005s	19.98%
Greedy + GA	4.8496	29.10	6/6	15.2%	1.1673s	0.00%
Greedy + SA	4.7462	28.48	6/6	15.2%	0.1836s	2.13%

Table 1: Comparative analysis under low traffic scenario

Scenario 2: Medium Traffic (23 Vehicles, 48 Spaces)

Method	Avg Rating	Total Rating	Assigned	Util %	Runtime	MILP Gap
MILP (B&B)	4.4993	103.48	23/23	48.5%	60.0005s	0.00%
SSP (B&B)	4.2509	97.77	23/23	48.5%	60.0004s	5.52%
Greedy Search	4.4755	102.94	23/23	48.5%	0.0066s	0.53%
Random	3.9995	87.99	22/23	57.6%	0.0010s	14.97%
Greedy + GA	4.4993	103.48	23/23	48.5%	2.1111s	0.00%
Greedy + SA	4.4755	102.94	23/23	48.5%	0.2116s	0.53%

Table 2: Comparative analysis under medium traffic scenario

Scenario 3: High Traffic (44 Vehicles, 48 Spaces)

Method	Avg Rating	Total Rating	Assigned	Util %	Runtime	MILP Gap
MILP (B&B)	4.0643	178.83	44/44	87.9%	60.0018s	0.00%
SSP (B&B)	3.9765	174.97	44/44	90.9%	60.0007s	2.16%
Greedy Search	4.0456	178.00	44/44	87.9%	0.0099s	0.46%
Random	3.8075	163.72	43/44	97.0%	0.0023s	8.45%
Greedy + GA	4.0685	179.01	44/44	87.9%	3.6408s	-0.10%
Greedy + SA	4.0456	178.00	44/44	87.9%	0.3766s	0.46%

Table 3: Comparative analysis under high traffic scenario

Discussion

The Classical MILP optimization achieved high satisfaction but scales exponentially, hitting the 60-second timeout in medium and high traffic scenarios. Interestingly, in Scenario 3, the Genetic Algorithm (GA) tuned online greedy search out-performed the Classical MILP (yielding a negative gap of -0.10%) since Classical MILP timed out and returned its best-found-so-far solution, while the GA-tuned online solver converged to a better assignment strategy globally in just 3.64 seconds.

The Random Assignment solver performs the worst across all metrics, showing a large optimality gap of 8.5% to 20% and failing to allocate all cars to spaces under medium/high traffic constraints (leaving cars unassigned).

The SSP (Branch & Bound) formulation represents state-space modeling but takes longer to prune branches due to probabilistic estimates, hitting the 60-second limit in all scenarios.

Optimizing weights with Genetic Algorithms and Simulated Annealing allows online Greedy Search to execute instantly (under 0.01 seconds) during live deployment while providing near-optimal ratings.

Future Work

Future improvements may include:

- Reinforcement learning approaches,
- Dynamic traffic prediction and reservation-aware parking systems,

- Multi-objective weight optimization,
- Adaptive real-time policy adjustments.